



파머완 8장 - 텍스트 분석

01. 텍스트 분석 이해

텍스트 분석 : 비정형인 텍스트를 분석하는 것

- 피처 벡터화 (feature vectorization) & 피처(특성) 추출 (feature extraction) : 텍스트를 단어 기반 다수의 피처로 추출하고 피처에 단어 빈도수와 같은 숫자 값을 부여하여 단어의 조합인 벡터값으로 표현

대표적 벡터화 변환 방법 : BOW(Bag of Words), Word2Vec

- 텍스트 분석 수행 프로세스
 1. 텍스트 전처리
 2. 피처 벡터화/추출
 3. 머신러닝 모델 수립 및 학습/예측/평가

파이썬 기반 NLP, 텍스트 분석 패키지 : NLTK, Genism, SpaCy

⇒ 다양한 텍스트 분석 적용 시 NLP 전용 패키지와 라이브러리를 결합하여 작성

02. 텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

[텍스트 정규화]

- 클렌징 : 텍스트에서 분석에 오히려 방해가 되는 불필요한 문자, 기호 등을 사전에 제거하는 작업
- 텍스트 토큰화 : 문서에서 문장을 분리하는 문장 토큰화, 문장에서 단어를 토큰으로 분리하는 단어 토큰화 등

#문장 토큰화 실습 : NLTK - sent_tokenize

스톱 워드 제거

: 스톱 워드 (분석에 큰 의미가 없는 단어) 제거 하는 전처리 작업

- Stemming : 원형 단어로 변환 시 일반적인 방법을 적용하거나 더 단순화된 방법을 적용해 원래 단어에서 일부 철자가 훼손된 어근 단어를 추출하는 경향

- Lemmatization : 품사와 같은 문법적인 요소와 더 의미적인 부분을 감안해 정확한 철자로 된 어근 단어를 찾아줌

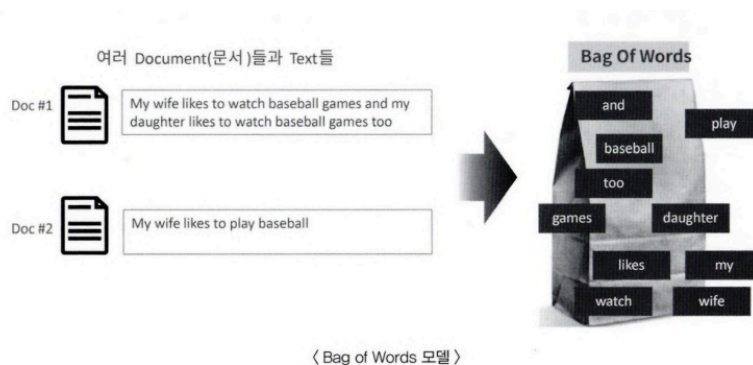
```
from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')

lemma = WordNetLemmatizer()
print(lemma.lemmatize('amusing', 'v'), lemma.lemmatize('amuses', 'v'), lemma.lemmatize('amused', 'v'))
print(lemma.lemmatize('happier', 'a'), lemma.lemmatize('happiest', 'v'))
print(lemma.lemmatize('fancier', 'a'), lemma.lemmatize('fanciest', 'a'))
```

→ NMF도 SVD와 유사하게 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링에 잘 사용되며, 추천 영역에 활발하게 적용됨

03. Bag of Words - BOW

- BOW 모델 = 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 일괄적으로 단어에 대해 빈도값을 부여해 피쳐값을 부여하는 모델



[단점]

- 문맥 의미 반영 부족, 희소 행렬 문제

[BOW 피쳐 벡터화]

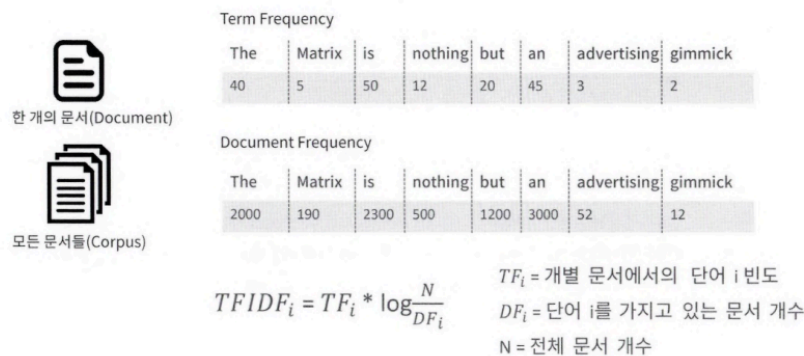
- 피쳐 벡터화 (텍스트는 특정 의미를 가지는 숫자형 값인 벡터 값으로 변환 필요) ⇒ 모든 문서에서 모든 단어를 칼럼 형태로 나열하고 각 문서에서 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터 세트 모델로 변경하는 것
- 카운트 벡터화 : Count를 부여하는 경우 → 카운트 값이 높을수록 중요한 단어

- TF-IDF (Term Frequency Inverse Document Frequency) 벡터화 : 개별 문서에서 자주 나타나는 단어에 높은 가중치를 주되, 모든 문서에서 전반적으로 자주 나타나는 단어에 대해서는 페널티를 주는 방식으로 값을 부여

→ 문서마다 텍스트가 길고 문서의 개수가 많은 경우 카운트 방식보다는 TF-IDF 방식을 적용하는 것이 좋다.

CountVectorizer : 카운트 기반의 벡터화를 구현한 클래스 → 피처 벡터화 + 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리도 함께 수행

max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
max features	추출하는 피처의 개수를 제한하며 정수로 값을 지정
stop_words	스톱 워드로 지정된 단어는 추출에서 제외
n_gram_range	n_gram 범위를 설정- 튜플 형태로 (범위 최솟값, 범위 최댓값)을 지정
analyzer	피처 추출을 수행한 단위를 지정 (디폴트 : word)
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용



[BOW 벡터화를 위한 희소 행렬]

희소 행렬 : 대규모 행렬의 대부분의 값을 0이 차지하는 행렬

- BOW 형태를 가진 언어 모델의 피처 벡터화는 대부분 희소 행렬 → 너무 많은 불필요한 0 값이 메모리 공간에 할당되어 메모리 공간이 많이 필요하며, 행렬의 크기가 커서 연산 시에도 데이터 액세스를 위한 시간이 많이 소모된다는 단점 존재

희소 행렬 - COO 형식 : 0이 아닌 데이터만 별도의 데이터 배열에 저장하고, 그 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식 → Scipy

희소 행렬 - CSR 형식 : COO 형식의 단점인 반복적인 위치 데이터 사용 문제점 해결 → 행 위치 배열 내에 있는 고유한 값의 시작 위치만 다시 별도의 위치 배열로 가지는 변환 방식

#실습

```
from scipy import sparse

dense2 = np.array([[0,0,1,0,0,5],
                  [1,4,0,3,2,5],
                  [0,6,0,3,0,0],
                  [2,0,0,0,0,0],
                  [0,0,0,7,0,8],
                  [1,0,0,0,0,0]])

# 0 아닌 데이터 추출
data2 = np.array([1, 5, 1, 4, 3, 2, 5, 6, 3, 2, 7, 8, 1])

# 행 위치와 열 위치를 각각 array로 생성
row_pos = np.array([0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5])
col_pos = np.array([2, 5, 0, 1, 3, 4, 5, 1, 3, 0, 3, 5, 0])

# COO 형식으로 변환
sparse_coo = sparse.coo_matrix((data2, (row_pos, col_pos)))

# 행 위치 배열의 고유한 값의 시작 위치 인덱스를 배열로 생성
row_pos_ind = np.array([0,2,7,9,10,12,13])

# CSR 형식으로 변환
sparse_csr = sparse.csr_matrix((data2, col_pos, row_pos_ind))

print('COO 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_coo.toarray())
print('CSR 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_csr.toarray())
```

05. 감성 분석 (sentiment analysis)

- 감성 분석 : 문서의 주관적인 감성/의견/감정/기분 등을 파악하기 위한 방법 → 텍스트의 단어와 문맥을 기반으로 감성 수치를 계산

- 지도 학습 방식 - 학습 데이터와 타겟 레이블 값을 기반으로 감성 분석 학습을 수행한 뒤 이를 기반으로 다른 데이터의 감성 분석을 예측하는 방법으로 일반적인 텍스트 기반의 분류와 거의 동일
- 비지도 학습 방식 - "Lexicon"이라는 감성 어휘 사전을 이용해서 긍정적, 부정적 감성 여부를 판단

[지도학습 기반 감성 분석 실습 - IMDB 영화평]

- Pipeline 객체 이용
- Count 벡터화
- TF-IDF 벡터화

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
```

```
# 스톱 워드는 English, ngram은 (1, 2)로 설정해 CountVectorization 수행
# LogisticRegression의 C는 10으로 설정
```

```
pipeline = Pipeline([
    ('cnt_vect', CountVectorizer(stop_words = 'english', ngram_range = (1, 2))),
    ('lr_clf', LogisticRegression(solver = 'liblinear', C = 10))
])
```

```
pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
pred_probs = pipeline.predict_proba(X_test['review'])[:, 1]
```

```
print('예측 정확도는 {0:.4f}, ROC_AUC는 {1:.4f}'.format(accuracy_score(y_test, p
                                roc_auc_score(y_test, pred_probs)))
```

```
pipeline = Pipeline([
    ('tfidf_vect', CountVectorizer(stop_words = 'english', ngram_range = (1, 2))),
    ('lr_clf', LogisticRegression(solver = 'liblinear', C = 10))
])
```

```

pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
pred_probs = pipeline.predict_proba(X_test['review'])[:, 1]

print('예측 정확도는 {0:.4f}, ROC_AUC는 {1:.4f}'.format(accuracy_score(y_test, p
                                roc_auc_score(y_test, pred_probs)))

```

[비지도학습 기반 감성 분석 소개]

- Lexicon 기반 → 감성 사전 - 긍정 감성, 부정 감성의 정도를 의미하는 수치인 감성 지수를 가짐
- WordNet = 시맨틱 분석을 제공하는 어휘 사전

→ 구성된 개별 단어를 Synset을 이용해 표현

- 예측 성능이 좋지 못하다는 단점

SentiWordNet 을 이용한 감성 분석

1. 문서(Document)를 문장(Sentence) 단위로 분해
2. 다시 문장을 단어(Word) 단위로 토큰화하고 품사 태깅
3. 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
4. senti_synset 객체에서 긍정 감성/부정 감성 지수를 구하고 이를 모두 합산해 특정 임계치 값 이상일 때 긍정 감성으로, 그렇지 않을 때는 부정 감성으로 결정

VADER를 이용한 감성 분석

- VADER = 소셜 미디어의 감성 분석 용도로 만들어진 룰 기반의 Lexicon
- VADER를 이용하면 감성 분석이 쉬움

```

from nltk.sentiment.vader import SentimentIntensityAnalyzer

senti_analyzer = SentimentIntensityAnalyzer()
senti_scores = senti_analyzer.polarity_scores(review_df['review'][0])
print(senti_scores)

```

```

def vader_polarity(review, threshold=0.1):
    analyzer = SentimentIntensityAnalyzer()
    scores = analyzer.polarity_scores(review)

    agg_score = scores['compound']
    final_sentiment = 1 if agg_score >= threshold else 0
    return final_sentiment

review_df['vader_preds'] = review_df['review'].apply( lambda x : vader_polarit
y_target = review_df['sentiment'].values
vader_preds = review_df['vader_preds'].values

print(confusion_matrix(y_target, vader_preds))
print("정확도:", np.round(accuracy_score(y_target, vader_preds),4))
print("정밀도:", np.round(precision_score(y_target, vader_preds),4))
print("재현율:", np.round(recall_score(y_target, vader_preds),4))

```